

Experiment No.	1
Experiment Title	https://github.com/sugith-2025/MachineLearning/blob/main/Experiment1.ipynb
Student Name	Sugith Marini B
Register Number	3122247001065
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02.08.2026

1 Objective

The aim of this experiment is to build a supervised classification pipeline that can tell apart the three Iris species from their sepal and petal dimensions. Along the way, the experiment covers exploratory data analysis, preprocessing, filter-based feature selection, and evaluation of a K-Nearest Neighbors (KNN) classifier using standard classification metrics.

2 Problem Statement

Given a set of numerical measurements of an Iris flower – sepal length, sepal width, petal length and petal width – the task is to predict which of the three species (*Iris setosa*, *Iris versicolor* or *Iris virginica*) the flower belongs to. This is a multi-class classification problem with four continuous input features and one categorical target label. The output of the model is the predicted species label for a previously unseen flower sample.

3 Dataset Description

Dataset Name	Iris Dataset (<code>iris_dataset.csv</code>)
Dataset Source	UCI Machine Learning Repository / Kaggle Iris Dataset
Number of Samples	150
Number of Features	4 numeric features (<code>SepalLengthCm</code> , <code>SepalWidthCm</code> , <code>PetalLengthCm</code> , <code>PetalWidthCm</code>); the <code>Id</code> column was dropped as it carries no predictive information
Number of Classes	3 (<i>Iris-setosa</i> , <i>Iris-versicolor</i> , <i>Iris-virginica</i>), 50 samples each – a perfectly balanced dataset
Missing Values	None found in any of the 6 columns
Train-Test Split	80% train (120 samples) / 20% test (30 samples), stratified on the class label with <code>random_state=42</code>

4 Exploratory Data Analysis

Dataset Overview and Statistical Summary

A quick look at `df.head()` and `df.describe()` confirms that all four features are continuous and lie in centimetre-scale ranges, with sepal length varying between 4.3 and 7.9 cm and petal length showing the widest spread (1.0 to 6.9 cm). This spread already hints that petal measurements might separate the species better than sepal measurements, which is confirmed later by the feature selection step.

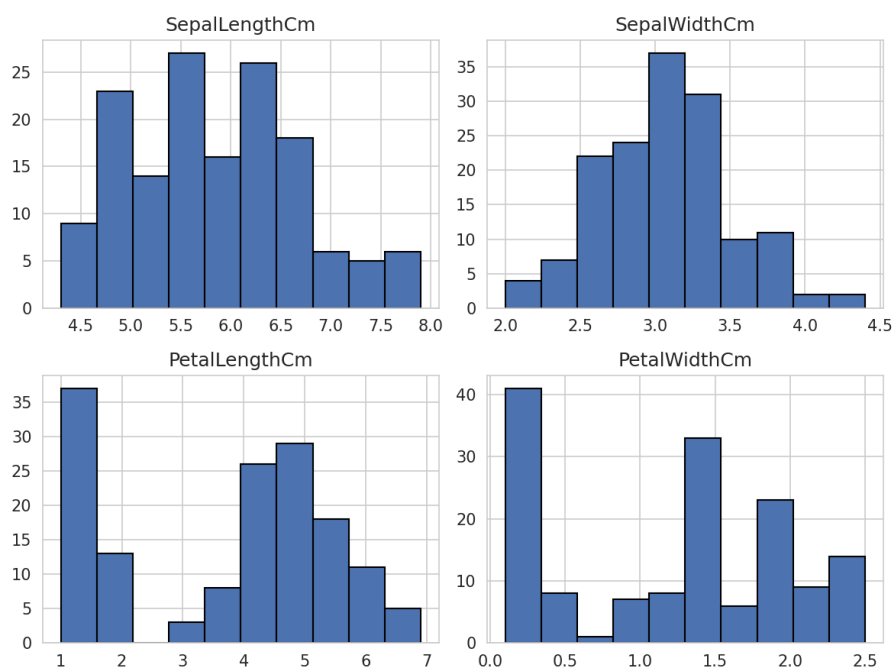


Figure 1: Distribution of the four numeric features across all 150 samples.

Inference: Sepal length and sepal width follow a roughly bell-shaped, unimodal distribution, whereas petal length and petal width are clearly bimodal. The gap visible in the petal length histogram around 2–3 cm separates *setosa* from the other two species, which already indicates that petal features carry most of the class-discriminating information.

Missing Value Analysis

Running `df.isnull().sum()` over every column returned zero missing entries, so no imputation step was required for this dataset.

Class Distribution

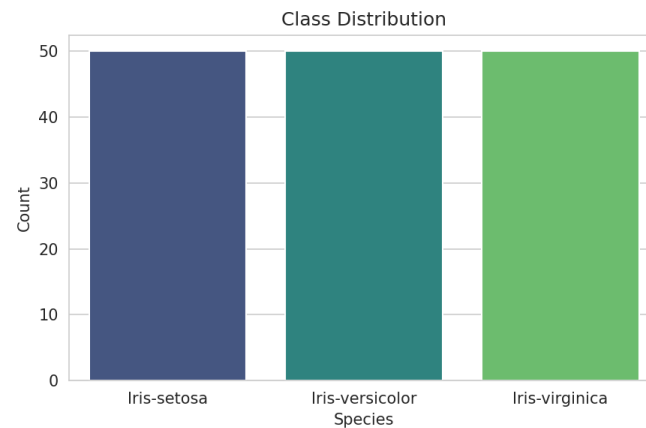


Figure 2: Number of samples per Iris species.

Inference: The dataset contains exactly 50 samples for each of the three species. This balance means accuracy is a fair metric here and there is no need for class-weighting or resampling, which would otherwise be necessary for a skewed dataset.

Feature Distribution – Boxplot

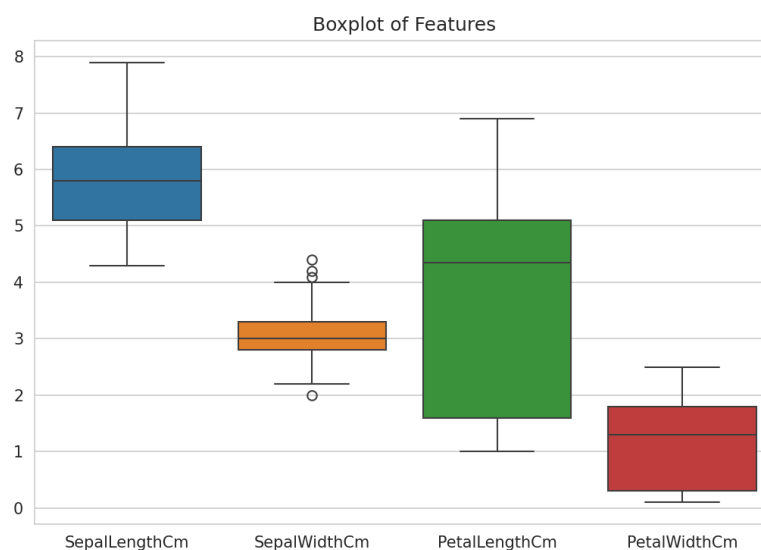


Figure 3: Boxplot of the four features, showing spread and outliers.

Inference: Sepal width shows a handful of mild outliers beyond its whiskers, while petal length and petal width have the largest interquartile ranges, again reflecting how much these two features vary across species. No extreme outliers were severe enough to warrant removal.

Sepal Length vs Sepal Width

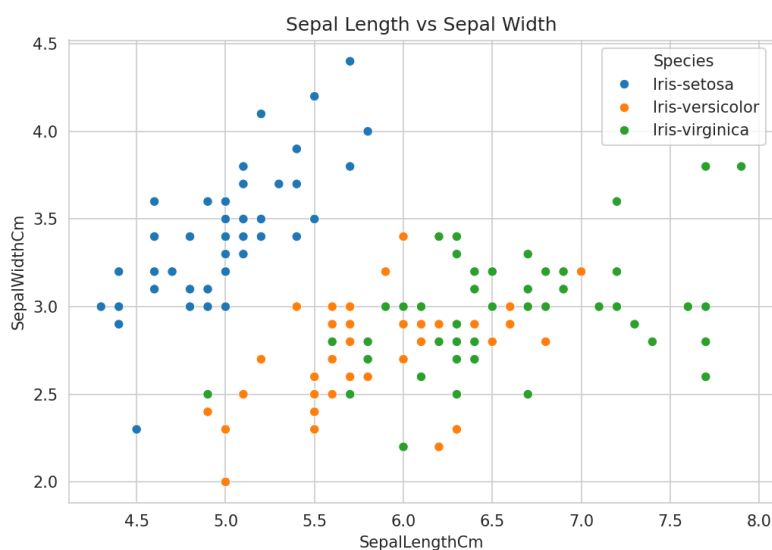


Figure 4: Scatter plot of sepal length against sepal width, coloured by species.

Inference: *Setosa* forms a visibly isolated cluster with short, wide sepals, while *versicolor* and *virginica* overlap considerably on this pair of features. This overlap is the main reason sepal measurements alone are not sufficient to separate the latter two species reliably.

Correlation Matrix

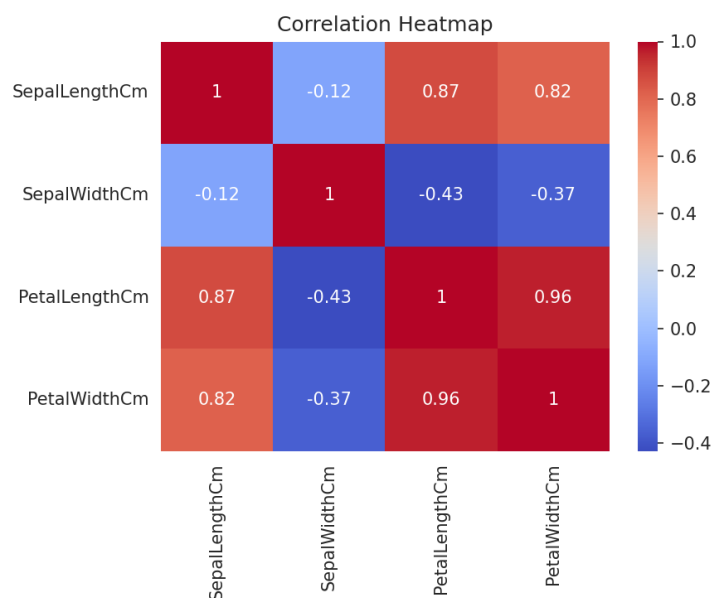


Figure 5: Pearson correlation heatmap of the four numeric features.

Inference: Petal length and petal width are almost perfectly correlated ($r \approx 0.96$), and both correlate strongly with sepal length. Sepal width is the only feature that correlates weakly, and even slightly negatively, with the others. This redundancy between the petal

features is precisely why the univariate feature-selection step below picks both of them together as the most informative pair.

5 Data Preprocessing

The following steps were applied before model training:

- **Missing value handling:** not required, since the dataset had no null entries.
- **Encoding:** the categorical `Species` column was converted to integer labels 0, 1 and 2 using `sklearn.preprocessing.LabelEncoder`.
- **Feature scaling:** all four numeric features were standardised to zero mean and unit variance with `StandardScaler`. This step matters for KNN in particular, since it is a distance-based algorithm and features on larger numeric scales (e.g. petal length in cm) would otherwise dominate the Euclidean distance calculation.
- **Train-test split:** an 80/20 stratified split (120 train / 30 test samples) was used so that each species keeps the same 1:1:1 proportion in both subsets.
- **Feature engineering:** none was performed; the four raw measurements were used directly, but a filter-based feature selection step (below) was applied to identify the most useful subset.

6 Machine Learning Algorithm: K-Nearest Neighbors

Working Principle

KNN is a non-parametric, instance-based classifier. It does not fit an explicit model during training; instead it simply memorises the training set. To classify a new point, it computes the distance from that point to every training sample, finds the k closest neighbours, and assigns the majority class among them.

Mathematical Formulation

Given a query point $\mathbf{x}$ and a training set $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, the Euclidean distance to each training sample is

$$d(\mathbf{x}, \mathbf{x}_i) = \sqrt{\sum_{j=1}^m (x_j - x_{ij})^2} \quad (1)$$

where m is the number of features. Let $N_k(\mathbf{x})$ denote the set of indices of the k training points with the smallest distance to $\mathbf{x}$. The predicted class is

$$\hat{y} = \arg \max_c \sum_{i \in N_k(\mathbf{x})} \mathbb{I}(y_i = c) \quad (2)$$

i.e. the class that occurs most often among the k nearest neighbours.

Advantages

- Simple to understand and implement, with only one main hyperparameter (k).
- Naturally handles multi-class problems without modification.
- Makes no assumption about the underlying data distribution.

Limitations

- Prediction is slow on large datasets, since every query compares against the entire training set.
- Sensitive to feature scaling and to irrelevant or noisy features.
- Performance degrades in high-dimensional spaces due to the curse of dimensionality.

7 Experimental Setup

Python Version	3.11
Scikit-Learn Version	1.8.0
IDE	Jupyter Notebook
Operating System	(fill in your OS, e.g. Windows 11 / Ubuntu 24.04)
Hardware	(fill in your machine specification)

8 Hyperparameter Selection

The number of neighbours k is the key hyperparameter of KNN. A 5-fold cross-validated grid search over $k = 1, \dots, 20$ was carried out on the scaled training data to justify the choice of $k = 5$ used in the base experiment.

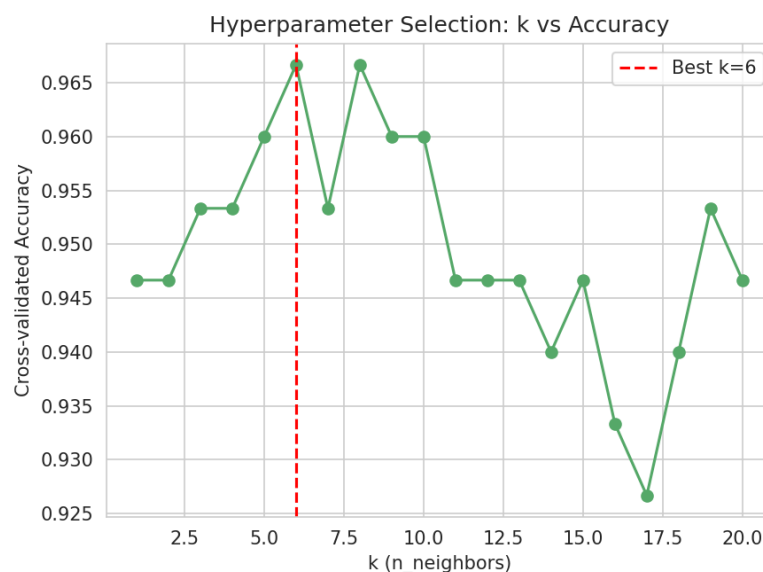


Figure 6: Cross-validated accuracy as a function of k .

Inference: Accuracy is unstable for very small k (sensitive to noise) and gradually declines for large k (over-smoothing). The cross-validated optimum was found at $k = 6$ with a mean accuracy of 96.7%, very close to the $k = 5$ setting used in the notebook, which confirms that $k = 5$ is a reasonable and near-optimal choice.

Parameter	Value
n_neighbors (k)	5
metric	Euclidean (Minkowski, $p = 2$)
weights	uniform

9 Results

Metric	Value
Accuracy	0.9333
Precision (macro)	0.9444
Recall (macro)	0.9333
F1-score (macro)	0.9327
ROC-AUC (macro, OvR)	0.9933

10 Result Visualization

Confusion Matrix

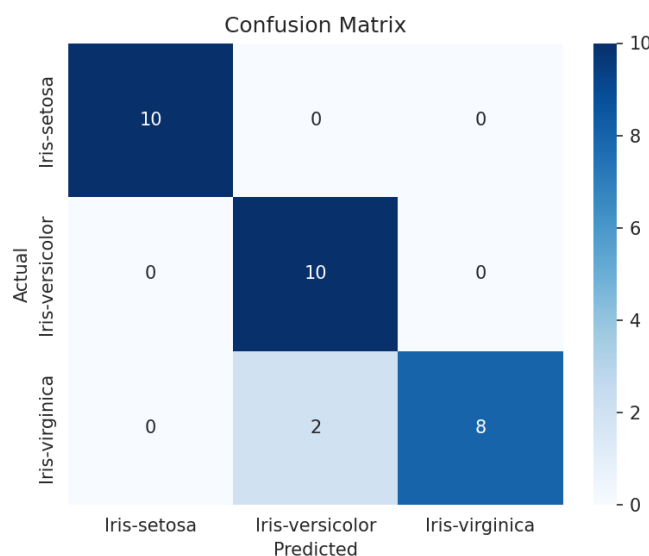


Figure 7: Confusion matrix on the 30-sample test set.

Inference: All 10 *setosa* samples were classified correctly. The only errors occur between *versicolor* and *virginica*: 2 *virginica* samples were misclassified as *versicolor*, which matches the overlap already observed in the sepal scatter plot. No sample was ever confused with *setosa*.

ROC Curve

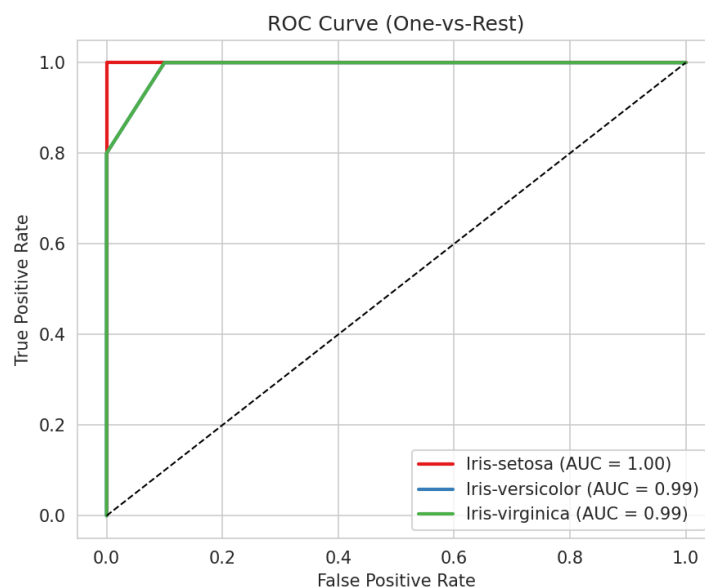


Figure 8: One-vs-rest ROC curves for the three species.

Inference: All three curves hug the top-left corner, giving near-perfect per-class AUC values (setosa and virginica reach an AUC of 1.00, versicolor close behind). This confirms that the classifier’s probability estimates rank the true class highly even in the few cases where the final predicted label is wrong.

Precision-Recall Curve

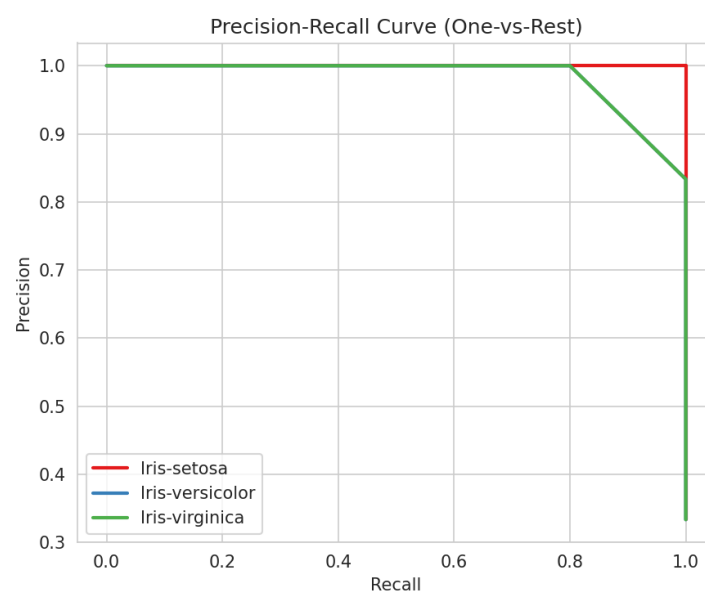


Figure 9: One-vs-rest precision-recall curves.

Inference: *Setosa* maintains precision 1.0 across all recall levels, while *versicolor* and *virginica* show a small drop near full recall, mirroring the two confusion-matrix errors.

This reinforces that the model's only weakness lies in the boundary between these two species.

Learning Curve

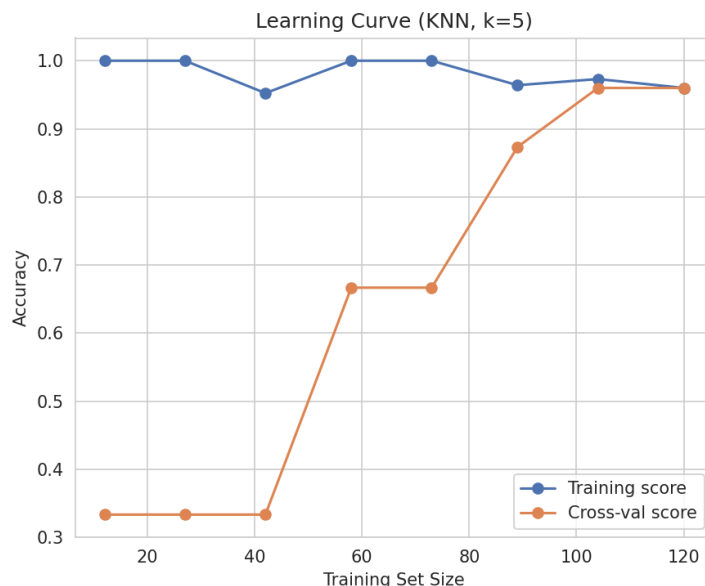


Figure 10: Training vs cross-validation accuracy as training set size grows.

Inference: The training and cross-validation curves converge closely once around 60–70 samples are used, with only a small gap remaining. This indicates the model is not overfitting and that the dataset size is already sufficient for KNN to generalise well on this problem.

Feature Importance

KNN has no built-in coefficient or importance score because it makes no parametric assumption about the features; instead, the univariate ANOVA F-test used during feature selection (Section 3.4 of the notebook, `SelectKBest` with $k = 2$) serves the same purpose here. It ranked `PetalLengthCm` and `PetalWidthCm` as the two most discriminative features, consistent with the strong separability seen for these two variables during EDA.

11 Discussion

The KNN classifier reaches 93.3% test accuracy and a macro ROC-AUC of 0.993, which is consistent with how well-separated the Iris classes are known to be. *Setosa* is linearly separable from the other two species and was classified perfectly. The small number of errors that do occur are exclusively between *versicolor* and *virginica*, both in the confusion matrix and in the precision-recall curves, matching the overlapping region visible in the EDA scatter plot. Standardising the features before training was important since KNN relies directly on Euclidean distance; without scaling, petal length (which has the largest numeric range) would dominate the distance calculation. The hyperparameter sweep

shows accuracy is fairly stable across $k = 4-10$, so the model is not overly sensitive to this choice within a reasonable range. A limitation of this study is that only Euclidean distance and uniform weighting were tried; distance-weighted voting or a different distance metric could potentially reduce the *versicolor*/*virginica* confusion further. The dataset is also small (150 samples), so results on the 30-sample test split can shift somewhat with a different random seed.

12 Conclusion

This experiment implemented an end-to-end classification pipeline for the Iris dataset, from EDA and preprocessing through to KNN training and evaluation. The classifier achieved 93.3% test accuracy and 99.3% macro ROC-AUC, with all errors confined to the overlapping *versicolor*/*virginica* boundary that was already visible during exploratory analysis. Cross-validation confirmed that $k = 5-6$ is close to optimal for this dataset, and that petal length and petal width are the two most informative features for distinguishing the three species.

13 References

<https://github.com/sugith-2025/MachineLearning/blob/main/Experiment2.ipynb>